Evaluating Snapshot-Consistent Live Migration for Stateful Containers in Kubernetes: A CRIU and Sidecar-Free eBPF Case Study

Musa Khasmammadov
Department of Computer Engineering & Cloud Systems
Baku, Azerbaijan
musaxasmammedov77@gmail.com

Abstract—While stateless microservices in Kubernetes achieve high availability through multi-replica deployments, rolling updates, and PodDisruptionBudgets, stateful singleton workloads—such as active session brokers, in-memory stream processors, and stateful caching nodes—cannot be trivially replicated without substantial distributed consensus overhead or intrusive application refactoring. When cluster nodes undergo maintenance or ephemeral cloud infrastructure (e.g., Amazon Web Services EC2 Spot instances) is reclaimed via a two-minute preemption notification, the standard Kubernetes scheduler executes a graceful teardown (SIGTERM followed by SIGKILL), destroying all volatile in-memory heap state.

This paper designs, implements, and evaluates a snapshot-consistent live migration architecture for stateful Kubernetes workloads. The pipeline coordinates the Linux Checkpoint/Restore in Userspace subsystem (CRIU v3.16.1) via the Kubelet Checkpoint API (KEP-2008), OCI artifact packaging via Buildah, the CRI-O container runtime (v1.31), and a sidecar-free Cilium eBPF Ingress architecture. We systematically evaluate the engineering trade-off between client-observed cutover blackout (Recovery Time Objective, RTO) and snapshot state drift (Recovery Point Objective, RPO).

Under a continuous 50 Hz (±20 ms) probing evaluation on AWS EC2, the decoupled migration pipeline achieves a cutover blackout window of 797–1078 ms (a 16.4%–38.2% reduction compared to native cold reschedule at 1289 ms). When accounting for the initial memory serialization freeze stutter ($T_{\text{freeze}} \approx$ 147–159 ms), cumulative client-perceived disruption spans 944–1237 ms (a 4.0%–26.8% improvement over cold restart). We document an empirical state drift of ≈ 696 ms ($\Delta S = 6$ uncommitted updates under a 10 Hz tick rate), analyze the risks of observable time rollback in transactional applications, evaluate memory scaling limits relative to the AWS Spot 120-second preemption deadline, and release our implementation as open-source.

Index Terms—Kubernetes, Container Live Migration, CRIU, CRI-O, Cilium eBPF, Stateful Workloads, Spot Instances, Recovery Point Objective.

I. Introduction

Container orchestration platforms, led by Kubernetes, rely on the premise of microservice idempotency and disposable instances [1]. In stateless deployments, node drain operations (kubectl drain) and instance preemptions are handled transparently: the control plane provisions replacement pods prior to terminating old replicas via rolling update strategies.

However, an important class of cloud-native workloads relies on long-running, in-memory volatile state:

1) In-Memory Session Brokers and State Engines: Streaming analytics engines (e.g., Apache Flink task slots), distributed game session coordinators, and caching nodes maintain gigabytes of volatile heap allocations.
2) Real-Time Inference and Aggregation Pipelines: Sensor ingest pipelines and windowed aggregators retain state in volatile memory to avoid database round-trip serialization.

When these stateful singleton workloads run on ephemeral cloud instances—most notably Amazon Web Services (AWS) EC2 Spot instances, which offer 70–90% cost reductions relative to On-Demand pricing but are subject to preemption with a strict 120-second notice [2], [3]—standard eviction results in complete state loss. When a node is drained, the kubelet issues a SIGTERM signal; upon expiration of terminationGracePeriodSeconds (or immediately upon Spot preemption deadlines), a SIGKILL signal destroys the address space, resetting the application to pristine zero state.

Historically, engineering teams prevent state loss by offloading volatile variables to distributed key-value storage (e.g., Redis clusters) [4]. While viable, this paradigm introduces three critical trade-offs: (i) serialization and network latency overheads (hundreds of microseconds across nodes compared to ≈ 100 ns for local L3 CPU cache/memory bus); (ii) complex application-level refactoring of existing monolithic and legacy binaries; and (iii) substantial infrastructure overhead for high-availability database clustering.

To avoid application refactoring while supporting infrastructure mobility, container live migration has emerged as an attractive alternative [5], [6]. In this paper, we explore the integration of checkpoint-restore mechanisms natively within Kubernetes. Our primary contributions are:

- Decoupled Migration Architecture: We design an automated 9-step orchestration pipeline combining KEP-2008 checkpointing [8], CRI-O, and Buildah to migrate containers across AWS EC2 physical

instances without application modifications.
- Formalization of RTO vs. RPO Trade-offs: We demonstrate that decoupling memory dumping from pod deletion minimizes client-observed cutover downtime ($T_{\text{blackout}} \in [797, 1078]$ ms), but introduces a bounded state drift (RPO $\approx$ 696 ms), formalizing the distinction between freeze latency, cutover blackout, and cumulative disruption.
- Analysis of Service Mesh Topologies: We evaluate how sidecar-free eBPF architectures (Cilium) resolve CRIU socket serialization crashes caused by in-pod loopback proxies in traditional sidecar meshes.
- Scalability and Spot Feasibility Modeling: We model multi-stage migration latency across heap capacities (26 MB to 8 GB) relative to the AWS 120-second termination window and analyze memory scaling bottlenecks.

II. System Architecture

A. Pipeline Overview

The migration architecture spans five foundational components: the Linux Checkpoint/Restore in Userspace subsystem (CRIU v3.16.1, packaged in Ubuntu 24.04 LTS), the Kubelet Checkpoint API (KEP-2008) [8], the CRI-O container runtime (v1.31), Buildah for OCI packaging, and a sidecar-free Cilium eBPF Ingress architecture.

B. Ingress Routing vs. L4 Socket Boundaries

When a pod migrates from k8s-worker-1 to k8s-worker-2, its Pod IP changes (e.g., from 10.0.1.45 to 10.0.2.112). If external clients connected directly to the Pod IP via plain L4 TCP, the 4-tuple (src_ip, src_port, dst_ip, dst_port) would be invalidated, triggering kernel RST packets.

Our architecture routes external traffic through an L7 Ingress Gateway (Cilium Ingress / Envoy). The ingress proxy terminates external client TCP/HTTP sessions. During migration:

- External client connections remain established at the ingress proxy.
- During the cutover blackout window (T_{blackout}), incoming requests are queued or retried by Envoy (configured with 3 retries on connect failure with a 25 ms backoff).
- As soon as the target pod completes CRI-O restore and passes readiness probes, Cilium's eBPF datapath updates the ingress endpoint table, directing buffered traffic to the new Pod IP.

C. Service Mesh Topologies: Sidecar vs. Sidecar-Free

In traditional service mesh architectures (e.g., Istio), an Envoy proxy runs as a sidecar container inside the application pod namespace. Traffic is intercepted via iptables PREROUTING rules over the 127.0.0.1 loopback interface.

When CRIU attempts to checkpoint a pod containing an active Envoy sidecar, two failure modes arise:

1) Loopback Socket Entanglement: CRIU cannot serialize active TCP connections bound to 127.0.0.1 when the remote target host cannot guarantee identical socket descriptors and peer namespaces.
2) Namespace-Locked Routing State: Pod-level iptables NAT state and inter-container UNIX domain sockets cannot be serialized and restored independently of the application process.

A sidecar-free topology (Cilium eBPF Service Mesh) [9] mitigates this issue by hosting Envoy at the node level. eBPF socket programs hook into socket events at the host cgroup boundary, avoiding namespace-locked proxies. Consequently, the application pod namespace contains only the isolated target process, enabling clean CRIU serialization.

III. Mathematical Latency, Drift, and Scalability Model

A. Latency Decomposition: Disruption vs. Blackout

Client-observed service disruption during migration consists of two distinct phases:

$$T_{\text{disruption}} = T_{\text{freeze}} + T_{\text{blackout}} \tag{1}$$

- Freeze Stutter (T_{freeze}): During T_{dump}, CRIU attaches to the process via PTRACE_SEIZE and halts thread execution while dumping memory pages from /proc/PID/pagemap. Client requests arriving during this window are queued in TCP socket buffers, experiencing a transient latency spike.
- Cutover Blackout (T_{blackout}): The elapsed time between source pod termination (t_{kill}) and verified successful response from the target pod (t_{ready}):

$$T_{\text{blackout}} = t_{\text{ready}} - t_{\text{kill}} = T_{\text{restore}} + T_{\text{route_sync}} \pm \delta_{\text{probe}} \tag{2}$$

where $\delta_{\text{probe}} = 20$ ms represents the sampling discretization of the 50 Hz probing harness.

The total end-to-end orchestration duration is:

$$T_{\text{total}} = T_{\text{dump}} + T_{\text{build}} + T_{\text{push}} + T_{\text{restore}} \tag{3}$$

B. The RTO vs. RPO Trade-off

The migration orchestrator can operate in two distinct modes:

1) Mode A: Decoupled Snapshot (Evaluated): The source container resumes execution immediately after memory dumping (KeepRunning: true) while Buildah packages and pushes the image ($T_{\text{build}} + T_{\text{push}}$).

This minimizes client blackout to $T_{\text{blackout}} \approx T_{\text{restore}}$ (797–1078 ms), but introduces state drift (uncommitted updates):

$$\text{RPO} = \Delta t_{\text{drift}} = t_{\text{kill}} - t_{\text{dump}} \approx T_{\text{build}} + T_{\text{push}} \approx 696 \text{ ms} \tag{4}$$

The number of uncommitted updates ΔS lost upon target cutover is bounded by:

$$\Delta S = \int_{t_{\text{dump}}}^{t_{\text{kill}}} \lambda(t)\, dt \approx \lambda \cdot (T_{\text{build}} + T_{\text{push}}) \tag{5}$$

where λ is the state mutation rate.

The 9-Stage Stateful Container Migration Sequence:
1. Trigger: Node maintenance or an AWS Spot Interruption Notice received via EC2 Instance Metadata Service (IMDS).
2. Memory Freeze & Dump: Kubelet Checkpoint API invokes CRIU via POST /checkpoint/boutique/pod/container. CRIU attaches via ptrace(PTRACE_SEIZE), halts execution threads, dumps address space to pages-1.img, and resumes source execution ($T_{\text{dump}} = T_{\text{freeze}} \approx 147$–$159$ ms).
3. OCI Packaging: Buildah embeds checkpoint.tar into a standard container layer tagged with CRI-O restore annotations ($T_{\text{build}} \approx 385$–$388$ ms).
4. Registry Push: The OCI image is pushed to the cluster-internal registry at 10.10.1.10:5000 ($T_{\text{push}} \approx 308$–$310$ ms).
5. Cutover Initiation: The source pod is terminated via the Kubernetes API; client-observed cutover blackout begins.
6. Target Scheduling: A target Pod manifest referencing the checkpoint OCI digest is scheduled onto k8s-worker-2.
7. CRI-O Restore: The CRI-O runtime reads the checkpoint annotation and invokes CRIU restore, mapping pages back into host RAM ($T_{\text{restore}} \approx 798$–$1080$ ms).
8. Process Resumption: The target process re-binds port :8090 with its restored heap and register state.
9. Ingress Retargeting: Cilium eBPF L7 Ingress detects endpoint readiness and retargets upstream proxying to the new target Pod IP.

Fig. 1. Detailed architectural sequence of the snapshot-consistent stateful live migration pipeline.

2) Mode B: Synchronous Stop-and-Copy (Theoretical Reference): The source container is paused before dumping and never resumed. This achieves zero state loss (RPO $\equiv 0$), but expands client blackout to the entire pipeline:

$$T_{\text{blackout}}^{\text{stop-and-copy}} = T_{\text{freeze}} + T_{\text{build}} + T_{\text{push}} + T_{\text{restore}} = T_{\text{total}} \tag{6}$$

Empirically, T_{total} spans 1642–1935 ms, which is 27.4%–50.1% worse than native cold reschedule (1289 ms). This establishes the fundamental trade-off: decoupling sacrifices ≈ 696 ms of state freshness to achieve sub-second client cutover blackout.

C. State Rollback and Phantom Data Risks

In Mode A, clients that received responses between t_{dump} and t_{kill} observed state that is subsequently rolled back upon target restoration. In our experiments, clients observed counter values reaching 60 (Run 1) and 85 (Run 2), but target execution resumed at 57 and 79, respectively. For transactional workloads, this represents an observable time rollback risk that requires idempotent transaction journals or external write-ahead logs (WAL) to resolve.

D. Analytical Scalability Model for AWS Spot Preemption

Live migration on AWS EC2 Spot instances must complete within the strict 120-second preemption notice:

$$T_{\text{total}}(M_{\text{RSS}}) < 120 \text{ seconds} \tag{7}$$

Unlike idealized models that assume constant packaging times, all four pipeline stages scale with resident set size (M_{RSS}):

$$T_{\text{total}}(M) = \frac{M}{B_{\text{IO}}} + \left(\frac{M}{B_{\text{tar}}} + t_{\text{oci}}\right) + \frac{M}{B_{\text{net}}} + \left(\frac{M}{B_{\text{restore}}} + t_{\text{runc}}\right) \tag{8}$$

where $B_{\text{IO}} \approx 125$ MB/s (EBS gp3 baseline), $B_{\text{net}} \approx 250$ MB/s (conservative sustained baseline for m7i-flex.large), $B_{\text{tar}} \approx 200$ MB/s, and $B_{\text{restore}} \approx 150$ MB/s. Table I summarizes the projected scaling behavior.

TABLE I
Analytical Scaling Projection vs. 120s Spot Deadline

Heap Size	T_{freeze}	T_{push}	Est. RPO	Est. T_{total}	Spot Feasibility
26 MB (Evaluated)	0.15 s	0.10 s	0.70 s	1.64 s	Validated (< 2% SLA)
256 MB	2.05 s	1.02 s	2.30 s	5.12 s	Safe (< 5% SLA)
1024 MB (1 GB)	8.19 s	4.10 s	9.22 s	20.5 s	Feasible (< 18% SLA)
4096 MB (4 GB)	32.8 s	16.4 s	36.9 s	81.9 s	Bound Limit (< 69% SLA)
8192 MB (8 GB)	65.5 s	32.8 s	73.7 s	163.8 s	Violates SLA (> 120 s)

Critically, Table I demonstrates that while heaps up to 4 GB satisfy the 120-second threshold, T_{freeze} and RPO grow substantially (> 8 s at 1 GB), confirming that single-pass dumping is ill-suited for large heaps without iterative pre-copying.

IV. Empirical Evaluation

A. Experimental Setup

The testbed was deployed on AWS EC2 in region eu-central-1 (Frankfurt) within a private VPC:

- Control Plane: 1x c7i-flex.large (2 vCPU, 4 GB RAM, Ubuntu 24.04 LTS, Linux 6.8).

- Worker Nodes: 2x m7i-flex.large (2 vCPU, 8 GB RAM).
- Background Contention: The full 11-microservice Google Online Boutique suite was continuously executed across the worker nodes to generate realistic CPU, memory, and network noisy-neighbor interference.
- Target Workload: An in-memory stateful service executing a 10 Hz memory heap counter.
- Probing Engine: An external HTTP prober querying the Cilium Ingress at 50 Hz (20 ms sampling intervals) to detect disruption windows.

B. Measured Results

Table II summarizes the empirical measurements recorded across native cold eviction and two multi-hop CRIU migration runs.

C. Detailed Analysis of Results

1) Cutover Blackout vs. Cumulative Disruption: In Run 2, the cutover blackout window was reduced to 797 ms (±20 ms), representing a 38.2% reduction relative to native cold reschedule (1289 ms). Across both runs, mean cutover blackout was 937.5 ms (a 27.2% improvement). However, when factoring in the freeze stutter ($T_{\text{freeze}} = 147$–159 ms), cumulative disruption was 944 ms (Run 2, a 26.8% improvement) and 1237 ms (Run 1, a 4.0% improvement).
2) RPO and State Drift Verification: In native eviction, the process state was entirely lost ($43544 \to 8$). Under CRIU migration, state continuity was preserved across nodes with a bounded rollback: $60 \to 57$ in Run 1 ($\Delta S = 3$) and $85 \to 79$ in Run 2 ($\Delta S = 6$). The measured loss of 6 updates in Run 2 is consistent with the theoretical prediction of equation (5) ($\lambda \cdot \Delta t_{\text{drift}} = 10 \times 0.696 \approx 7$). The lower loss in Run 1 ($\Delta S = 3$) reflects clock quantization and asynchronous pod teardown timing.
3) Measurement Resolution: The observation that $T_{\text{blackout}} = 797$ ms was marginally lower than CRI-O's logged $T_{\text{restore}} = 798$ ms falls well within the ±20 ms probing resolution.

V. Related Work

A. Virtual Machine and Container Live Migration

Live migration was formalized at the hypervisor layer by Clark et al. [7], utilizing iterative pre-copying of memory pages. In container virtualization, Stoyanov and Kollingbaum [5] evaluated CRIU-based migration bottlenecks, showing that network transfer of memory pages dominates migration time. Nadgowda et al. [6] developed Voyager to enable complete container migration across clouds. Reber [8] spearheaded the integration of checkpoint-restore mechanisms natively into Kubernetes via KEP-2008. Our study builds upon KEP-2008 by demonstrating end-to-end multi-node pod migration over production OCI registries.

B. Service Mesh Architectures

Traditional service meshes inject sidecars into application pod namespaces. Recent architectures, notably Cilium eBPF Service Mesh [9], migrate layer-7 routing to node-level proxies. We show that sidecar-free architectures eliminate in-pod loopback socket dependencies, resolving a primary impediment to container checkpointing.

VI. Threats to Validity and System Limitations

1) Exploratory Sample Size: The empirical results represent a dual-hop case study ($n = 2$ per path). While sufficient to demonstrate architectural viability and state continuity, large-scale statistical campaigns ($N \geq 50$) are necessary to establish rigorous confidence intervals and tail latencies ($p95, p99$).
2) State Rollback in Decoupled Mode: Mode A incurs an RPO of ≈ 696 ms. Applications requiring strict zero-loss must either accept stop-and-copy blackout (> 1.6 s) or implement application-level WAL.
3) Memory Scalability Bounds: Single-pass dumping is limited to workloads with moderate heap sizes ($\leq$ 4 GB). Multi-gigabyte heaps require iterative dirty-page pre-copying.
4) L4 Protocol Scope: Our evaluation utilized an HTTP-based stateful service behind an L7 ingress proxy. Workloads relying on direct long-lived L4 TCP or raw UDP connections require complementary connection migration mechanisms.

VII. Conclusion

This paper presented an empirical evaluation of snapshot-consistent live migration for stateful containers in Kubernetes using CRI-O, CRIU, and sidecar-free Cilium eBPF. The architecture achieves sub-second cutover blackout (797 ms) and 4.0%–26.8% lower cumulative disruption than cold restart, while preserving in-memory state with a bounded RPO (≈ 696 ms). Future work will incorporate iterative pre-copying to enable zero-RPO migration on multi-gigabyte stateful workloads.

Artifact Availability

All manifests, orchestration scripts, operator code, and benchmarking harnesses are publicly available at: https://github.com/musaxasmammedov7/final-research.

References

[1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016.
[2] Netflix Technology Blog, "Creating Your Own EC2 Spot Market," Netflix Inc., 2015.
[3] Amazon Web Services, "Spot Instance Interruptions," AWS Documentation, 2024.
[4] Redis Ltd., "Redis Latency Problems Troubleshooting," https://redis.io/docs/management/optimization/latency/, 2024.
[5] R. Stoyanov and M. J. Kollingbaum, "Efficient Live Migration of Linux Containers," in Proc. 13th Workshop on Virtualization in High-Performance Cloud Computing (VHPC), Springer, Cham, 2018, pp. 279–291.

TABLE II
Empirical Benchmark Comparison: Proposed CRIU Stateful Migration vs. Native Kubernetes Eviction

Execution Run	Pipeline Path	T_{total}	T_{freeze}	T_{build}	T_{push}	T_{blackout}	$T_{\text{disruption}}$	Counter Continuity
Native Eviction (Baseline)	worker-2 $\rightarrow$ worker-1	1289 ms	N/A	N/A	N/A	1289 ms	1289 ms	43544 $\rightarrow$ 8 (Total Reset)
CRIU Migration (Run 1)	worker-1 $\rightarrow$ worker-2	1935 ms	159 ms	385 ms	310 ms	1078 ms	1237 ms	51 $\rightarrow$ 60 $\rightarrow$ 57 ($\Delta S = 3$)
CRIU Migration (Run 2)	worker-2 $\rightarrow$ worker-1	1642 ms	147 ms	388 ms	308 ms	797 ms	944 ms	77 $\rightarrow$ 85 $\rightarrow$ 79 ($\Delta S = 6$)

[6] S. Nadgowda, C. Isci, and R. Koller, "Voyager: Complete Container Migration," in Proc. 37th IEEE Int. Conf. on Distributed Computing Systems (ICDCS), 2017, pp. 2137–2144.

[7] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live Migration of Virtual Machines," in Proc. 2nd USENIX Symp. on Networked Systems Design and Implementation (NSDI), 2005, pp. 273–286.

[8] A. Reber, "Forensic Container Checkpointing in Kubernetes," Kubernetes Official Blog / CNCF, Dec. 2022. [Online]. Available: https://kubernetes.io/blog/2022/12/05/forensic-container-checkpointing-alpha/

[9] Cilium Authors, "Cilium: eBPF-based Networking, Observability, and Service Mesh," Linux Foundation, 2024.

[10] Checkpoint/Restore In Userspace, "CRIU Architecture and Core Internals," https://criu.org, 2024.